

MARAN PMS — Fixes Módulo Administración

PROMPT: Caja Chica sin asiento + Cuenta contable en pago proveedor + Acceso OP + Bug emisión OP

FIX 1 — Gasto de caja chica: quitar cuenta contable, mantener centro de costo

Ubicación

client/src/pages/admin-caja.tsx — dentro del `MovimientoDialog`, en el bloque condicional que se muestra cuando `tipo === "egreso_gasto"`.

Cambio

Antes (muestra cuenta contable Y centro de costo para `egreso_gasto`):

```
{tipo === "egreso_gasto" && (  
  <>  
    <div className="space-y-1">  
      <Label>Cuenta contable</Label>  
      <Select value={cuentaContableId} onChange={setCuentaContableId}>  
        ...  
      </Select>  
    </div>  
    <div className="space-y-1">  
      <Label>Centro de costo</Label>  
      <Select value={centroCosto} onChange={setCentroCosto}>  
        ...  
      </Select>  
    </div>  
  </>  
)}
```

Después (solo muestra centro de costo, sin cuenta contable):

```

{tipo === "egreso_gasto" && (
  <div className="space-y-1">
    <Label>Centro de costo</Label>
    <Select value={centroCosto} onChange={setCentroCosto}>
      <SelectTrigger>
        <SelectValue placeholder="Opcional..." />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="recepcion">Recepción</SelectItem>
        <SelectItem value="restaurant">Restaurante</SelectItem>
        <SelectItem value="spa">Spa</SelectItem>
        <SelectItem value="administracion">Administración</SelectItem>
        <SelectItem value="mantenimiento">Mantenimiento</SelectItem>
        <SelectItem value="general">General</SelectItem>
      </SelectContent>
    </Select>
  </div>
)}

```

También limpiar el estado `cuentaContableId` **cuando el tipo cambia a** `egreso_gasto` :

```

// En el useEffect o handler de cambio de tipo:
if (nuevoTipo === "egreso_gasto") {
  setCuentaContableId("");
}

```

FIX 2 — Pago a proveedor: agregar cuenta contable Y centro de costo

Ubicación

Mismo componente `MovimientoDialog` , **agregar un bloque condicional para** `tipo === "egreso_proveedor"` .

Actualmente no hay ningún bloque para `egreso_proveedor` . **Agregar después del bloque de** `egreso_gasto` :

```

{tipo === "egreso_proveedor" && (
  <>
    <div className="space-y-1">
      <Label>Cuenta contable</Label>

```

```

<Select value={cuentaContableId} onChange={setCuentaContableId}>
  <SelectTrigger>
    <SelectValue placeholder="Seleccionar cuenta..." />
  </SelectTrigger>
  <SelectContent>
    {(cuentas as any[])
      .filter((c: any) =>
        c.tipo === "egreso" ||
        c.codigo?.startsWith("4") ||
        c.codigo?.startsWith("5") ||
        c.codigo?.startsWith("6")
      )
      .map((c: any) => (
        <SelectItem key={c.id} value={String(c.id)}>
          {c.codigo} - {c.nombre}
        </SelectItem>
      ))}
  </SelectContent>
</Select>
</div>
<div className="space-y-1">
  <Label>Centro de costo</Label>
  <Select value={centroCosto} onChange={setCentroCosto}>
    <SelectTrigger>
      <SelectValue placeholder="Opcional..." />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="recepcion">Recepción</SelectItem>
      <SelectItem value="restaurant">Restaurante</SelectItem>
      <SelectItem value="spa">Spa</SelectItem>
      <SelectItem value="administracion">Administración</SelectItem>
      <SelectItem value="mantenimiento">Mantenimiento</SelectItem>
      <SelectItem value="general">General</SelectItem>
    </SelectContent>
  </Select>
</div>
</>
)}

```

FIX 3 — Acceso rápido a Pago de Proveedores

Problema

Para pagar un proveedor hay que navegar: Administración → Proveedores Contables →

buscar el proveedor → Ver Comprobantes → Cuenta Corriente → Emitir OP. Es demasiado profundo.

Solución A — Botón en la página principal de Administración (/admin)

En `client/src/pages/admin.tsx` (o `administration.tsx`), agregar una card de acceso rápido junto a las existentes:

```
<Card
  className="cursor-pointer hover:shadow-md transition-shadow"
  onClick={() => navigate("/purchase-invoices?tab=pagos")}
>
  <CardContent className="pt-6">
    <div className="flex items-center gap-4">
      <div className="p-3 rounded-lg bg-green-50">
        <CreditCard className="h-6 w-6 text-green-600" />
      </div>
      <div>
        <h3 className="font-semibold">Pago de Proveedores</h3>
        <p className="text-sm text-muted-foreground">
          Emitir órdenes de pago y gestionar cuenta corriente
        </p>
      </div>
    </div>
  </CardContent>
</Card>
```

Solución B — Tab “Pagos” en /purchase-invoices

En `client/src/pages/purchase-invoices.tsx`, agregar un tab “Pagos” que muestre directamente la lista de proveedores con saldo pendiente (la vista de cuenta corriente consolidada), sin necesidad de entrar proveedor por proveedor.

```
<Tabs defaultValue={searchParams.get("tab") || "comprobantes"}>
  <TabsList>
    <TabsTrigger value="comprobantes">Comprobantes</TabsTrigger>
    <TabsTrigger value="pagos">Pagos a Proveedores</TabsTrigger> { /* ← NUEVO */ }
  </TabsList>

  <TabsContent value="pagos">
    <PagosProveedoresTab />
  </TabsContent>
</Tabs>
```

El componente `PagosProveedoresTab` muestra la lista de proveedores con facturas pendientes, con su saldo total adeudado y un botón “Emitir OP” que abre directamente el `PaymentOrderDialog`:

```
function PagosProveedoresTab() {
  const { data: proveedores = [] } = useQuery({
    queryKey: ["/api/accounting-suppliers/con-saldo"],
  });
  // Endpoint que devuelve suppliers que tienen facturas en estado 'pendiente'

  return (
    <div className="space-y-2">
      <div className="text-sm text-muted-foreground mb-4">
        Proveedores con facturas pendientes de pago
      </div>
      {proveedores.length === 0 && (
        <div className="text-center py-8 text-muted-foreground">
          No hay facturas pendientes de pago
        </div>
      )}
      {proveedores.map((prov: any) => (
        <div key={prov.id} className="flex items-center justify-between p-4 borde">
          <div>
            <div className="font-medium">{prov.razonSocial}</div>
            <div className="text-sm text-muted-foreground">
              {prov.cantFacturasPendientes} factura(s) – Total: ${prov.saldoPendi
            </div>
          </div>
          <Button onClick={() => openPaymentDialog(prov)}>
            <CreditCard className="h-4 w-4 mr-2" />
            Emitir OP
          </Button>
        </div>
      )})}
    </div>
  );
}
```

Nuevo endpoint necesario

```
// GET /api/accounting-suppliers/con-saldo
// Devuelve proveedores que tienen al menos una factura en estado 'pendiente'
app.get("/api/accounting-suppliers/con-saldo", requireAuth, async (req, res) => {
  const result = await db.execute(sql`
    SELECT
```

```

        s.id,
        s.razon_social,
        COUNT(pi.id) AS cant_facturas_pendientes,
        SUM(pi.monto_total) AS saldo_pendiente
    FROM accounting_suppliers s
    INNER JOIN purchase_invoices pi ON pi.supplier_id = s.id AND pi.estado = 'pen
    GROUP BY s.id, s.razon_social
    ORDER BY saldo_pendiente DESC
`);
res.json(result.rows);
});

```

FIX 4 — Bug al emitir Orden de Pago (error rojo)

Diagnóstico

El endpoint `POST /api/payment-orders` **valida:**

```
WHERE id = ANY(facturaIds) AND supplier_id = supplierId AND estado = 'pendiente'
```

Si `facturasRes.rows.length !== facturaIds.length` **devuelve error 400:** "Algunas facturas no son válidas o no están pendientes"

Causa probable: Las facturas de condición de pago "contado" se registran pero es posible que en algunos casos queden con `estado = 'pagado'` al crearse, o que el `supplierId` enviado desde el frontend no coincida exactamente con el `supplier_id` almacenado (diferencia de tipo: string vs integer).

Fix A — Corregir el tipo de `supplierId` en el frontend

En `PaymentOrderDialog`, al hacer el submit, asegurar que `supplierId` sea un número entero:

```

// En handleSubmit:
createMut.mutate({
  supplierId: Number(supplier?.id), // ← forzar a número
  fecha: form.fecha,
  facturaIds: selectedInvoices,      // ya son números (ids de las facturas)
  // ...resto igual
});

```

Fix B — Mejorar el mensaje de error para diagnóstico

Cambiar el error genérico para que muestre exactamente qué pasó:

```
// En el endpoint POST /api/payment-orders:
if (facturasRes.rows.length !== facturaIds.length) {
  // Identificar cuáles facturas fallaron
  const idsEncontrados = facturasRes.rows.map((r: any) => r.id);
  const idsFallidos = facturaIds.filter((id: number) => !idsEncontrados.includes(

  // Verificar si existen pero no están pendientes
  const estadosRes = await db.execute(sql`
    SELECT id, estado FROM purchase_invoices WHERE id = ANY(${facturaIds}::int[])
  `);

  const noEncontradas = facturaIds.filter(
    (id: number) => !estadosRes.rows.find((r: any) => r.id === id)
  );
  const noPendientes = estadosRes.rows
    .filter((r: any) => r.estado !== 'pendiente')
    .map((r: any) => `#${r.id} (${r.estado})`);

  let errorMsg = "No se pudo generar la OP: ";
  if (noEncontradas.length > 0) errorMsg += `Facturas no encontradas: ${noEncontradas.length}`;
  if (noPendientes.length > 0) errorMsg += `Facturas no pendientes: ${noPendientes.length}`;

  return res.status(400).json({ error: errorMsg });
}
```

Fix C — Agregar cuentas faltantes al plan de cuentas si no existen

Las cuentas 2.1.1.01 (Proveedores a Pagar), 1.1.1.01 (Caja) y 1.1.1.02 (Banco Macro) son necesarias para generar el asiento de la OP. El seed solo las inserta si la tabla está vacía.

Agregar una migración de inserción segura que se ejecuta siempre al iniciar el servidor (usa ON CONFLICT DO NOTHING):

```
// En server/index.ts o donde se inicializa la app, agregar:
await db.execute(sql`
  INSERT INTO accounting_accounts (codigo, nombre, tipo) VALUES
    ('1.1.1.01', 'Caja', 'activo'),
    ('1.1.1.02', 'Banco Macro', 'activo'),
    ('2.1.1.01', 'Proveedores a Pagar', 'pasivo')
  ON CONFLICT (codigo) DO NOTHING
`);
```

Esto asegura que aunque el seed no haya corrido completo, estas cuentas críticas siempre existan.

Fix D — El asiento de la OP no debe bloquear la creación

Actualmente el `try/catch` del asiento contable solo loguea el error pero no lo propaga — eso está bien. Verificar que sea así:

```
// Generar asiento contable — si falla, no bloquear la OP
try {
  const supplier = await db.execute(sql`SELECT razon_social FROM accounting_suppl
  const entryId = await generarAsientoOP({ ...op, supplier: supplier.rows[0] as a
  await db.execute(sql`UPDATE payment_orders SET asiento_id = ${entryId} WHERE id
} catch (ae) {
  console.error("Error generando asiento OP (no crítico):", ae);
  // NO hacer return ni throw — la OP se creó correctamente igual
}
```

Si el `try/catch` ya estaba así pero el error igual llega al usuario, significa que el error ocurre **antes** del bloque de asiento — es decir en la validación de facturas (Fix A/B). Aplicar Fix A primero.

NOTAS

1. El orden de aplicación recomendado es: **Fix A → Fix C → Fix B → Fix D**. El Fix A (tipo del `supplierId`) es el más probable causante del error visible.
2. No modificar la lógica de retenciones ni el cálculo del `totalAbonado` — esa parte funciona correctamente.
3. Los fixes 1 y 2 (caja chica) son puramente de UI, no requieren cambios en el backend ni en la base de datos.